동적 params와 Suspense, 프리렌더가 막히는 조건

- 날짜: 2026-10-30
- 대상: Next.js 16.4.3, `lgcns-fe-next`
- 설정: `next.config.ts`의 `cacheComponents: true`, `partialPrefetching: true`
- 블로그와 상품 모두 `params`를 `<Suspense>` 안에서 `await`한다.

`cacheComponents`가 켜지면 Next.js는 각 라우트가 정적 껍데기를 먼저 그릴 수 있는지 개발 중에 검사한다. `params`는 요청 URL이 있어야 값이 정해지는 런타임 데이터다. 그 값을 `<Suspense>` 밖에서 `await`하면 껍데기 전체를 그 값까지 기다리게 되어 Blocking Route가 된다.

설치된 가이드 `node_modules/next/dist/docs/01-app/02-guides/migrating-to-cache-components.md`의 "Await params inside Suspense"가 이 규칙을 적는다. `generateStaticParams`가 없는 라우트는 `params` 프로미스를 `<Suspense>` 안으로 넘기고, 그 안에서 `await`한다.

1. 두 동적 라우트

파일	세그먼트	params 타입	await 위치	<Suspense>
`app/(main)/blog/[blogId]/page.tsx`	한 칸, `[blogId]`	`Promise<{ blogId: string }>`	`BlogContent` 안	페이지가 `fallback={<p>Loading...</p>}`로 감쌈
`app/(main)/product/[...productCategoryName]/page.tsx`	catch-all	`Promise<{ productCategoryName: string[] }>`	`ProductContent` 안	페이지가 `fallback={<p>Loading...</p>}`로 감쌈

`(main)`은 라우트 그룹이다. URL에 들어가지 않는다. 실제 경로는 `/blog/이름`과 `/product/분류/하위`다.

`app/(main)/layout.tsx`는 `params`를 받지 않는다. `Header`를 그린 뒤 `children`을 둔다. 동적 값 대기는 레이아웃이 아니라 각 `page`에 있다.

두 페이지 모두 `generateStaticParams`를 내보내지 않는다. `export const instant = false`도 없다. 그래서 공식 문서의 "껍데기를 프리렌더하려면 Suspense 안에서 await" 경로에 해당한다. `instant = false`는 그 검사를 잠시 끄고 라우트가 블로킹해도 되게 하는 표시이지, 데이터를 캐시하거나 정적 경로를 만들지는 않는다.

2. 블로그 라우트. await가 Suspense 안에 있다

`app/(main)/blog/[blogId]/page.tsx`는 동기 함수다. `params`를 기다리지 않고 프로미스 그대로 넘긴다.

```
export default function BlogPage({
  params,
}: {
  params: Promise<{ blogId: string }>
}) {
  return (
    <Suspense fallback={<p>Loading...</p>}>
      <BlogContent params={params} />
    </Suspense>
```

```
  )
}
```

component/templates/BlogContent.tsx가 비동기 함수이고, 여기서 프로미스를 푼다.

```
export default async function BlogContent({
  params,
}: {
  params: Promise<{ blogId: string }>
}) {
  const { blogId } = await params
  return (
    <div>
      <h1>Blog Page</h1>
      <p>{blogId}</p>
    </div>
  )
}
```

이전에 페이지 함수가 먼저 `const { blogId } = await params`를 하고, 그 다음에 `<Suspense>`를 반환했다. 경계는 이미 값이 정해진 뒤에 그려지므로, `await`는 경계 밖이다. 개발 서버는 그때 이 메시지를 냈다.

> Next.js encountered runtime data during prerendering.

문서의 Before 예와 같다. `await`를 `BlogContent`로 내린 형태가 문서의 After 예와 같다. 페이지 함수는 동기라서 껍데기와 `Loading...`을 먼저 반환하고, `blogId` 문단은 그 값이 정해진 뒤 스트리밍된다.

`/blog/name`이면 `blogId`는 문자열 `"name"`이다. 동적 세그먼트 한 칸은 `string`이다.

3. 상품 라우트. await가 Suspense 안에 있다

app/(main)/product/[...productCategoryName]/page.tsx는 catch-all이다. 폴더 이름 `[...productCategoryName]`의 `...`가 뒤 경로를 배열로 모은다. 페이지 함수는 동기이고, 배열은 component/templates/ProductContent.tsx가 푼다.

```
export default function ProductPage({
  params,
}: {
  params: Promise<{ productCategoryName: string[] }>
}) {
  return (
    <Suspense fallback={<p>Loading...</p>}>
      <ProductContent params={params} />
    </Suspense>
  )
}
```

```
export default async function ProductContent({
  params,
}: {
  params: Promise<{ productCategoryName: string[] }>
}) {
  const { productCategoryName } = await params
  console.log(productCategoryName.join("/"))
```

```
  return (
    <div>
      <CustomTitle TitleType="h1" className="text-2xl font-bold">
        {productCategoryName.join("/")}
      </CustomTitle>
      <p>{productCategoryName}</p>
    </div>
  )
}
```

설치된 동적 라우트 문서의 표와 대응하면 이렇다.

URL	productCategoryName
/product/laptop	["laptop"]
/product/laptop/gram	["laptop", "gram"]
/product	매칭되지 않음. 이 폴더는 optional catch-all [[...productCategoryName]]가 아니다

join("/")은 배열을 슬래시 문자열로 만든다. /product/laptop/gram의 제목 자식은 "laptop/gram"이다. console.log는 서버 컴포넌트 렌더 때 서버 터미널에 같은 문자열을 찍는다. 브라우저 전용 스크립트가 아니다.

이 await는 <Suspense> 안이다. 페이지 함수는 Loading...을 먼저 반환하고, 제목은 배열이 정해진 뒤 스트리밍된다. 이전에 페이지 함수가 반환 전에 await params를 하던 형태는 블로그가 Blocking Route를 내던 때와 같은 접근이었다. instant = false는 없다.

제목은 component/atoms/CustomTitle.tsx다. TitleType="h1"이라 렌더 결과는 h1이고, 클래스는 twMerge("text-2xl font-bold text-center text-white", "text-2xl font-bold")다. 두 문자열 모두 소스에 완성된 Tailwind 클래스로 적혀 있다. productCategoryName으로 ` text-${...} `를 만들지는 않는다.

4. 한 칸 세그먼트와 catch-all

표기	이 저장소	값
[blogId]	app/(main)/blog/[blogId]	string 하나. /blog/a/b는 이 페이지에 안 맞음
[...productCategoryName]	app/(main)/product/[...productCategoryName]	string[]. /product 자체는 안 맞음
[[...name]]	없음	파라미터 없는 경로까지 포함하고, 없으면 undefined

Next.js 16에서 페이지가 받는 params는 프로미스다. 두 페이지 모두 Promise<{ ... }>로 타입을 적고, 그 프로미스를 자식에 넘긴다. await는 BlogContent와 ProductContent에 있다. 동기 객체로 바로 구조 분해하지 않는다.

5. 로딩 시점

이 라우트에는 <script src>가 없다. Next가 서버에서 페이지 함수를 실행하고 HTML을 보낸다.

1. cacheComponents와 partialPrefetching 때문에 개발 서버는 라우트 껍데기를 프리렌더할 수 있는지 본다.
2. 블로그 페이지 함수는 await가 없다. Loading...이 있는 Suspense를 바로 반환한다. BlogContent의 await params는 그 경계 안이라, blogId 문단만 요청 시점에 이어 붙는다.
3. 상품 페이지 함수도 await가 없다. ProductContent의 await params는 그 경계 안이라, 제목과 문단만 요청 시점에 이어 붙는다.

4. `(main)` 레이아웃의 `Header`는 이 대기와 별도로 먼저 그려질 수 있는 부모다. 페이지가 막히면 그 페이지 내용이 레이아웃 뒤에서 기다린다.

`cookies()`, `headers()`, `searchParams`도 같은 검사 대상이다. 이 두 페이지는 그것을 호출하지 않는다. 걸린 런타임 데이터는 `params`뿐이다.

6. 공식 문서

설치본 경로와 공개 문서가 같다.

- Await params inside Suspense. 로컬 파일 `node_modules/next/dist/docs/01-app/02-guides/migrating-to-cache-components.md`
- blocking-prerender-runtime
- instant 세그먼트 설정. `export const instant = false`는 블로킹을 허용하는 표시
- cacheComponents
- Dynamic Routes. `[segment]`, `[...slug]`, `[[...slug]]`, `params` 타입
- React Suspense